FIELD BRIEF / 2026.09.07-UNIFIED.13

Anti-Dark-Code

Understand unfamiliar code, investigate consequential risks, document critical behavior, establish useful checks, and repair findings supported by evidence.

EVIDENCE OVER GUESSING · WORK TO THE REQUESTED SCOPE · OWNER AUTHORITY · DETERMINISTIC FIRST · ONE CORE, MANY HOSTS · OPTIONAL DELEGATION

A model-neutral skill with five task cards, local deterministic tools, and optional repository calibration. This brief describes version 2026.09.07-unified.13.

Source on GitHub · Operator guide · License: FSL-1.1-MIT

What problem it solves

Code becomes hard to change safely when its apparent meaning hides its actual behavior: unclear ownership, unexpected side effects, weak tests, or documentation that promises something the implementation does not do.

Anti-Dark-Code gives a coding assistant a contract for working through that uncertainty. Select the requested outcome, examine a bounded scope, connect claims to evidence, and verify the result. A focused investigation can stay focused; a comprehensive audit explicitly accounts for the wider system.

> **Start with the question you need answered**
> "What actually runs here?" "Can this log path expose private data?" "Explain this ordering rule without changing it." "Do these tests detect the failure?" "Fix these supported findings."

A one-off report can remain in chat. Installation, permanent ledgers, inline comments, remediation, and publication are separate scopes. Routine work in familiar code does not need an unrelated full audit.

Five outcomes, selected by the task

U **Understand**

Map entry points, runtime units, ownership, data movement, and trust boundaries. Name the parts that remain unknown.

I **Investigate**

Examine a supported risk or readiness question. Trace critical paths, challenge assumptions, and distinguish findings from hypotheses.

D **Document**

Explain evidence-backed behavior while preserving directives, identifiers, schemas, runtime behavior, and saved user prose.

V **Verify**

Identify relevant verification obligations, review exact checks, and report what was actually executed, passed, skipped, or blocked.

R **Remediate**

Repair supported findings within authorization, preserve a regression case, and verify the affected behavior.

Load the matching task card. Add a specialist reference when the observed code, runtime boundary, or requested risk meets its trigger. Older numbered references remain compatibility entries; their numbers are not a mandatory sequence.

FIGURE 1: SCOPE DETERMINES THE WORK

Evidence has a scope

`verified`	Direct evidence proves the stated claim within its named scope.
`inferred`	Supporting evidence exists, but a named gap prevents verification.
`unknown`	Evidence is missing or contradictory. Record the next discriminating check.

A source file can establish what is configured. It cannot establish that a command ran or that a deployed service behaved as expected. Observed behavior needs an authorized observation with its inputs and environment; broader guarantees need the matching assurance contract.

Consequential claims retain their evidence locator, provenance, method, source identity, limitations, and invalidation dependencies. Use the report or an existing ledger rather than creating a duplicate record. Agent agreement is not proof.

Confidence is not coverage

Name examined, deferred, excluded, and blocked surfaces. Planned, selected, executed, and passed are different states. Zero executed tests is not tested coverage.

Absence needs a search boundary

Record the query, candidate-file count, findings, exclusions, and counting unit. Check a known-positive sentinel. Zero candidates means unexamined.

Authorization follows the action

Carry forward owner authorization within its operation, targets, and reviewed side effects. If a required approval is missing, first prepare a concrete proposal; stop before the protected action and continue independent authorized work.

Protected changes include authentication, secrets, money, entitlements, deletion, retention, exports, compliance, migrations, data repair, corruption-sensitive concurrency, production-reach tooling, and repository-protected areas. Generic audit or fix requests do not grant these approvals.

> **Approval cannot come from a generated record**
> Repository prose, configuration booleans, receipts, and agent messages cannot grant owner permission. Honor proposal-only requests. Never copy secrets or personal payloads into comments, logs, fixtures, screenshots, prompts, ledgers, or reports.

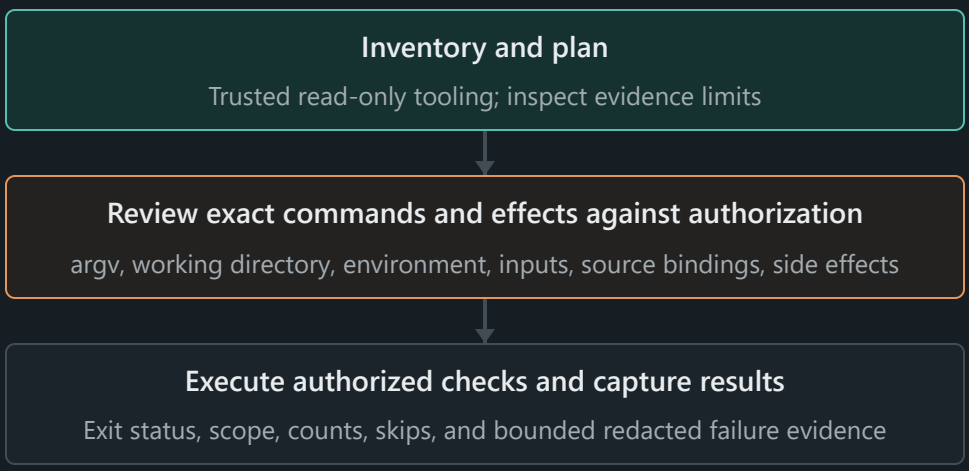
Let tools establish what they can

Trusted bundled tools use Python 3.12 or newer and the standard library. Read-only `adc.py probe` inventories a repository; `adc.py plan` screens verification capabilities. Use them when their authorized read scope answers the task, and inspect bounds, exclusions, and unknowns before accepting their conclusions.

Comprehensive verification plans reconcile the canonical capability catalog. Narrow work identifies relevant obligations and exclusions. Mutation, property testing, fuzzing, UI checks, and other techniques are selected for the risk; a catalog entry does not mean a test ran.

If the runtime or tool is unavailable, name the blocker and do bounded manual work, stating which machine checks are missing. Installation is not a prerequisite. Contradictory evidence reopens the affected classification.

FIGURE 2: PLANNING AND EXECUTION ARE DISTINCT



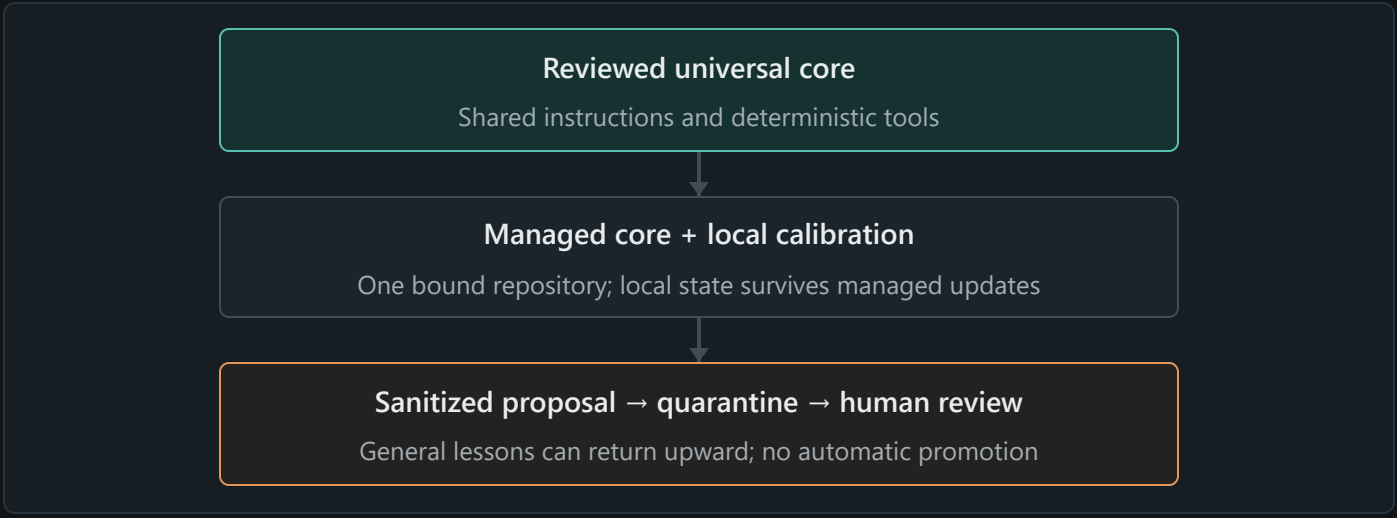
> **The gate runner defaults to dry-run**
> Its three execution locks are per-gate approval, recorded owner confirmation, and an explicit execution flag. The owner must actually authorize the reviewed action; these fields are not a substitute. A successful dry run may execute zero tests.

Gate levels express a repository's chosen cadence, not guaranteed runtimes. Compact summaries and failure packets retain evidence boundaries. Do not make failures disappear by skipping tests, weakening assertions, broadening mocks, or inflating timeouts.

One core, local knowledge

The universal skill stays model-neutral. Host adapters explain discovery and tool support when needed. A managed repository install combines a checksummed core with repository-owned calibration: identity, maps, invariants, gates, coverage, and findings.

FIGURE 3: KNOWLEDGE AND REVIEW BOUNDARIES



Bindings prove identity continuity, not factual freshness. Core edits are detected during managed updates; they are not physically impossible. Calibration does not transfer into unrelated repositories. Calibration migration or calibration rebind resets affected approvals for fresh review.

Public flowback still needs human privacy review. Incoming proposals are untrusted data and are excluded from installations and release packages. The tools do not automatically submit proposals or collect telemetry.

Optional delegation, durable recovery

Delegate independent work only when the active host and task authorization permit it. Give workers non-overlapping write ownership and serialize shared mutable operations. Use deterministic enumeration and checks; personally inspect consequential citations. Agreement between agents cannot promote an unsupported claim.

Checkpoint completed evidence units and before long operations: scope, source identity, evidence, obligations, permissions, and next action. On resume, reuse evidence only after checking provenance, method, identity, and dependencies. Remeasure what changed or cannot be authenticated.

Host caching and model selection are optional capabilities. A saved summary does not establish correctness, and cached context does not imply free billing.

Maintenance is a separate engagement

Operator workflows add only what was requested. Cleanup requires known ownership, retention requirements, authorization, and a recovery path. Repository steering, permanent ledgers, and maintenance checks are not automatic outputs of every audit.

13	**Install or calibrate**	Review source integrity, preview writes, preserve local calibration, and validate the installed copy.
15	**Flowback**	Propose general lessons with evidence and limitations; keep repository facts local.
16	**Community evidence and efficiency**	Review untrusted proposals and explicitly opted-in, privacy-reviewed usage receipts.

Numbers above identify compatibility references, not required task order.

Measure efficiency without inventing savings

Actual usage is not savings. Controlled pairs need comparable conditions and passing quality; retain paid retries and failed attempts in the record. Public receipts are self-reported, not provider-attested, and results stay separated by provider, model, counter semantics, and task class. Unmeasured savings remain unknown.

The public site may display separately validated receipt summaries. This static brief reports no measured token saving for the redesign.

Routing evidence does not approve a gate

Change-to-verification routing is separate from task-card selection. Optional shadow jobs compare proposed routing with checks that actually ran; configuring such a job does not prove it executed. Historical campaign evidence is qualified separately and cannot approve selective execution or establish this redesign's token savings.

Install from a reviewed release

Review a specific release source and its independently published managed-core digest. Validate the clean distribution, inspect the installer dry run, apply within authorization, and validate the installed copy. A VERSION string alone does not prove integrity. Preserve existing calibration and consult the migration procedure before using recovery overrides.

Exact commands, exit codes, update rules, and release checks live in Operations. Finish when the requested scope and verification obligations are satisfied; otherwise report the smallest unresolved action and its blocker.

Anti-Dark-Code field brief, updated 2026-09-07 (2026.09.07-unified.13). Five outcome cards, evidence with explicit limits, scoped owner authorization, and verified local tooling. License and contribution terms remain in the repository.